

資料結構

Data Structure

Lab 11

姓名： 廖彥瑄

學號： 10810030

Lab11-Ex1.

Complement Level Sum function

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
const int EMPTY = NULL; // 用 NULL 代表沒有節點
class TreeNode {
public:
    int value; // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty() && i < arr.size()) {
            TreeNode** nodePtr = q.front(); // 取出 queue 的第一個節點指標
            q.pop(); // 將該節點從 queue 中刪除

            // 左子節點
            if (i < arr.size()) {
```

```

        if (arr[i] != EMPTY) {
            (*nodePtr)->left = new TreeNode(arr[i]); // 添加左子節點
            q.push(&((*nodePtr)->left)); // 將左子節點的指標加入 queue
        }
        i++;
    }
    // 右子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) {
            (*nodePtr)->right = new TreeNode(arr[i]); // 添加右子節點
            q.push(&((*nodePtr)->right)); // 將右子節點的指標加入 queue
        }
        i++;
    }
}
return root;
}

void Depth_first_search(TreeNode* node) { // 深度優先搜尋
    if (node == nullptr) return;
    cout << node->value << " "; // 訪問當前節點的值
    Depth_first_search(node->left); // 遞迴遍歷左子樹
    Depth_first_search(node->right); // 遞迴遍歷右子樹
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
    q.push(root); // 將根節點的指標加入 queue

    while (!q.empty()) {
        TreeNode* current = q.front(); // 取出 queue 的第一個節點指標
        q.pop(); // 將該節點從 queue 中刪除
        cout << current->value << " ";
        if (current->left) q.push(current->left); // 將左子節點的指標加入 queue
        if (current->right) q.push(current->right); // 將右子節點的指標加入 queue
    }
}
}

```

```

int Level_sum(int targetLevel){
    if(root==nullptr || targetLevel<0)return -1;//空的
    queue<TreeNode*>q;
    q.push(root);//從最上層開始
    int level=0;
    while(!q.empty()){
        int levelSize=q.size();//記錄這層有幾個節點
        int sum=0;
        for(int i=0;i<levelSize;i++){
            TreeNode*current=q.front();//取出最前面的節點
            q.pop();
            if(level==targetLevel)sum+=current->value;//節點加起來
            if(current->left)q.push(current->left);//左子節點放進 queue
            if(current->right)q.push(current->right);//右子節點放進 queue
        }
        if(level==targetLevel)return sum;
        level++;
    }
    return -1;//沒有這層
}

};

int main() {
    BinaryTree tree;
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL,
NULL };
    tree.buildTree(arr);

    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;

    int layer;
    cout<<"please enter the layer to query, starting from 0:";
    if(cin>>layer){
        int sum=tree.Level_sum(layer);
        if(sum== -1){
            cout<<"the layer exceeds the tree height";//沒有這層
        }
    }
}

```

```

    }else{
        cout<<"the sum of level "<<layer<<" is:"<<sum;//顯示該層總和
    }
}
cout<<endl;

system("pause");
return 0;
}

```

Result

```

C:\Users\User\Desktop\資料結構\Lab11\Lab11_Ex1.exe
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0:2
the sum of level 2 is:22
請按任意鍵繼續 . . . .

```

```

1 #include <iostream>
2 #include <queue>
3 #include <vector>
4 using namespace std;
5 const int EMPTY = NULL; // 用 NULL 代表沒有節點
6 class TreeNode {
7 public:
8     int value;// 節點值
9     TreeNode* left;// 左子節點
10    TreeNode* right;// 右子節點
11
12    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}//初始化節點
13
14 };
15 class BinaryTree {
16 public:
17     TreeNode* root;// 樹根節點
18
19     BinaryTree() : root(nullptr) {}
20
21     TreeNode* buildTree(const vector<int>& arr) {}// 建立樹
22     if (arr.empty() || arr[0] == EMPTY) return nullptr;
23
24     queue<TreeNode*> q;// 建立queue 儲存待處理的節點指標
25     root = new TreeNode(arr[0]);// 建立根節點 (陣列第一個元素)
26     q.push(&root);// 將根節點的指標加入queue
27
28     size_t i = 1;// 陣列索引
29     while (!q.empty() && i < arr.size()) {
30         TreeNode* nodePtr = q.front();// 取出queue的第一個節點指標
31         q.pop();// 將該節點從queue中刪除
32
33         // 左子節點
34         if (i < arr.size()) {
35             // 右子節點
36             if (i < arr.size()) {
37                 // ...
38             }
39         }
40     }
41 }
42
43 int main() {
44     vector<int> arr = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
45     BinaryTree tree;
46     tree.buildTree(arr);
47     int layer;
48     cout << "BFS Result: ";
49     tree.bfs();
50     cout << endl;
51     cout << "please enter the layer to query, starting from 0:";
52     int layer;
53     while (cin >> layer && layer < 0 || layer > 10) {
54         cout << "Invalid layer, please enter a layer between 0 and 10." << endl;
55     }
56     int sum;
57     tree.sumOfLevel(layer, sum);
58     cout << "the sum of level " << layer << " is:" << sum << endl;
59     system("pause");
60     return 0;
61 }

```

```
C:\Users\User\Desktop\資料結構\Lab11\Lab11_Ex1.exe
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0:4
the layer exceeds the tree height
請按任意鍵繼續 . . .
```

```
C:\Users\User\Desktop\資料結構\Lab11\Lab11_Ex1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globale)
Lab11_Ex1.cpp
1 #include <iostream>
2 #include <queue>
3 #include <vector>
4 using namespace std;
5 const int EMPTY = NULL; // 用 NULL 代表沒有節點
6 class TreeNode {
7 public:
8     int value; // 節點值
9     TreeNode* left; // 左子節點
10    TreeNode* right; // 右子節點
11
12    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
13 };
14
15 class BinaryTree {
16 public:
17     TreeNode* root; // 樹根節點
18
19     BinaryTree() : root(nullptr) {}
20
21     TreeNode* buildTree(const vector<int>& arr) { // 建立樹
22         if (arr.empty() || arr[0] == EMPTY) return nullptr;
23
24         queue<TreeNode*> q; // 建立queue 儲存待處理的節點指標
25         root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
26         q.push(&root); // 將根節點的指標加入queue
27
28         size_t i = 1; // 陣列索引
29         while (!q.empty() && i < arr.size()) {
30             TreeNode* nodePtr = q.front(); // 取出queue的第一個節點指標
31             q.pop(); // 將該節點從queue中刪除
32
33             // 左子節點
34             if (i < arr.size()) {
```

```
C:\Users\User\Desktop\資料結構\Lab11\Lab11_Ex1.exe
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
please enter the layer to query, starting from 0:4
the layer exceeds the tree height
請按任意鍵繼續 . . .
```